ShoeSwap: Browse Filters Tutorial

Saved for after the MVP. Adds filtering by brand, condition, size range and mileage to the browse page. About 30 to 40 minutes.

How it works

A filter form submitted with GET puts the choices in the URL (like `/browse/?brand=nike&condition=good`), and the view narrows the queryset to match. No extra packages needed.

1. Add a filter form to `shoes/forms.py`

```
class BrowseFilterForm(forms.Form):
    brand = forms.ChoiceField(required=False)
    condition = forms.ChoiceField(required=False)
    size_min = forms.DecimalField(
        label='Size from (US)', required=False, min_value=3, max_value=18,
        decimal_places=1, widget=forms.NumberInput(attrs={'step': '0.5'}),
    )
    size_max = forms.DecimalField(
        label='Size to (US)', required=False, min_value=3, max_value=18,
        decimal_places=1, widget=forms.NumberInput(attrs={'step': '0.5'}),
    )
    max_km = forms.IntegerField(label='Max distance (km)', required=False,
min_value=0)

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        # Reuse the choices from the model, with an "Any" option on top.
        brand_choices = Shoe._meta.get_field('brand').choices
        condition_choices = Shoe._meta.get_field('condition').choices
        self.fields['brand'].choices = [('', 'Any brand')] +
list(brand_choices)
        self.fields['condition'].choices = [('', 'Any condition')] +
list(condition_choices)

    def clean(self):
        cleaned = super().clean()
        low, high = cleaned.get('size_min'), cleaned.get('size_max')
```

```
        if low is not None and high is not None and low > high:
            raise forms.ValidationError('"Size from" can\'t be bigger than
"Size to".')
        return cleaned
```

Pulling the choices from the model means that when you add a brand later, the filter picks it up automatically.

2. Update the `browse` view in `shoes/views.py`

Import the new form:

```
from .forms import ShoeForm, BrowseFilterForm
```

Replace `browse`:

```
@login_required
def browse(request):
    """Show available shoes from other users, with optional filters."""
    shoes = (Shoe.objects
             .filter(is_available=True)
             .exclude(owner=request.user)
             .select_related('owner')
             .order_by('-date_added'))

    form = BrowseFilterForm(request.GET or None)
    if form.is_valid():
        data = form.cleaned_data
        if data['brand']:
            shoes = shoes.filter(brand=data['brand'])
        if data['condition']:
            shoes = shoes.filter(condition=data['condition'])
        if data['size_min'] is not None:
            shoes = shoes.filter(size_us__gte=data['size_min'])
        if data['size_max'] is not None:
            shoes = shoes.filter(size_us__lte=data['size_max'])
        if data['max_km'] is not None:
            shoes = shoes.filter(km_run__lte=data['max_km'])

    context = {'shoes': shoes, 'form': form}
    return render(request, 'shoes/browse.html', context)
```

Each filter only applies when the person filled that box in. `__gte` means greater than or equal, `__lte` means less than or equal. `request.GET or None` keeps the form blank on first visit.

3. Update `browse.html`

Add `{% load django_bootstrap5 %}` under the `extends` line, and put the filter panel at the top of the content block, above the `{% for %}` loop:

```
{% extends 'shoes/base.html' %}
{% load django_bootstrap5 %}

{% block page_header %}
<h2>Browse shoes</h2>
{% endblock page_header %}

{% block content %}
<form method="get" class="card card-body mb-4">
    {% if form.non_field_errors %}
        <div class="alert alert-danger">{{ form.non_field_errors }}</div>
    {% endif %}
    <div class="row g-2">
        <div class="col-md-4">{% bootstrap_field form.brand %}</div>
        <div class="col-md-4">{% bootstrap_field form.condition %}</div>
        <div class="col-md-4">{% bootstrap_field form.max_km %}</div>
        <div class="col-md-4">{% bootstrap_field form.size_min %}</div>
        <div class="col-md-4">{% bootstrap_field form.size_max %}</div>
    </div>
    <div>
        <button type="submit" class="btn btn-primary">Filter</button>
        <a href="{% url 'shoes:browse' %}" class="btn btn-outline-
secondary">Clear</a>
    </div>
</form>

<p class="text-muted">{{ shoes|length }} shoe{{ shoes|length|pluralize }}
found</p>

{# your existing {% for shoe in shoes %} ... {% empty %} ... {% endfor %}
stays below #}
{% endblock content %}
```

The `<form method="get">` has no `{% csrf_token %}`, which is correct. That tag is only for POST forms.

Change the `{% empty %}` message so it makes sense when filters are on:

```
{% empty %}
<p>No shoes match. Try widening your filters, or <a href="{% url
'shoes:browse' %}">clear them</a>.</p>
```

4. Test it

1. Open `/browse/` with a few shoes of different brands, sizes and conditions listed by the other account.
2. Pick a brand and click Filter. Only that brand should show, and the URL should now include `?brand=...`.
3. Combine filters, for example size from 9 to 10 with condition Good.
4. Enter size from 11 and size to 9. You should see the error message.
5. Click Clear and confirm all shoes are back.

Notes

- Choosing Other as the brand filter shows every Other shoe, whatever brand was typed in. Filtering by the typed names would need a free-text search box, which fits version 2 together with sorting by price.
- Finish swap requests first. This is a bonus, not part of the Friday MVP.